

litemall — Database Schema & Service Architecture

Shared `litemall-db` anatomy · table ownership per microservice · order-domain interactions · design principles

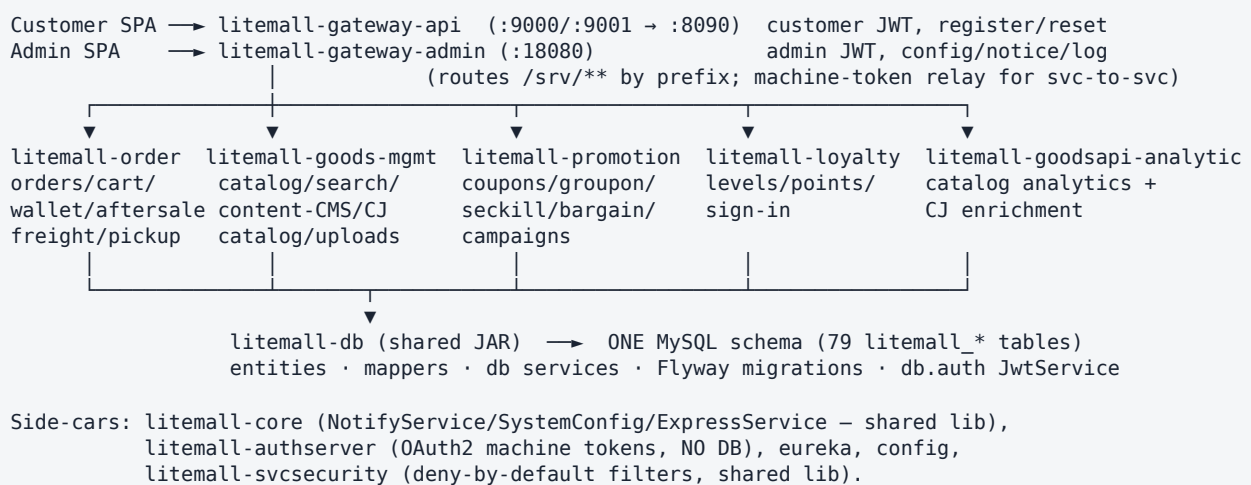
Generated 2026-07-15 from the live dev database (79 tables, Flyway V1-V37) and the source tree at `litemall/` (master, post-Wave-4).

Contents

1. System overview & architectural stance
2. The shared `litemall-db` module
3. Full table inventory, grouped by domain, with owners
4. Service → table ownership matrix
5. Shared tables and orphan (legacy) tables
6. Migration timeline (Flyway V1-V37)
7. Order domain deep dive
8. How other services interact with order tables
9. Design principles & rationale

1 · System overview & architectural stance

The platform is a microservices fork of litemall. The legacy monolith APIs (`litemall-wx-api`, `litemall-admin-api`) were decommissioned in Wave 4; traffic now flows through two edge gateways into domain services, all of which persist through one shared library, `litemall-db`, against a **single MySQL schema**.



The central design decision: this is **not** database-per-service. All services share one schema through one hand-maintained persistence library. Isolation is enforced *logically* — by table-ownership convention, anti-corruption facades over HTTP/Feign for cross-domain *writes*, and a small number of deliberate, documented shared-read exceptions (§8). This trades physical isolation for operational simplicity (one Flyway history, one backup unit, no distributed queries) — the classic "shared database, logical ownership" pattern.

2 · The shared `litemall-db` module

`litemall-db` is a library JAR (not a service) that every data-touching module depends on. It contains the domain entities, MyBatis mappers, thin "db services", Flyway migrations, and — unusually — the edge-

JWT primitives.

2.1 Two generations of persistence code

	Generated baseline (V1 era)	Hand-written fork additions (V3+)
Entities	34 entities, each with a *Example companion (e.g. LitemallGoods, LitemallOrder, LitemallUser)	~31 entities with no Example class (e.g. LitemallStore, LitemallEventOutbox, LitemallShippingTemplate, LitemallArticle, LitemallUserResetToken)
Mappers	34 Example-style mappers with generated XML (full CRUD + *ByExample)	~15 bespoke XML mappers (joins, aggregates, guarded updates: OrderMapper, StatMapper, FreightTemplateMapper, PageMapper, EventOutboxMapper ...) + ~18 annotation-only mappers (@Select / @Insert : refresh/reset tokens, wallet, loyalty, bargain/seckill...)
Convention	The MyBatis generator is frozen at the V1 baseline . The module is hand-maintained: new tables arrive only via Flyway migration + hand-written entity/mapper pairs. Never regenerate — regeneration would ignore the hand additions, and entity + mapper-XML must be edited <i>together</i> (a missed XML edit silently drops the column: the historic now_money / OrderMapper.xml failures).	

2.2 Structural quirks worth knowing

- **Custom SqlSessionFactory** (config/MyBatisConfig): declaring the bean makes Spring Boot's MyBatis autoconfig back off, so YAML mybatis.configuration.* is ignored; the config compensates by calling setMapUnderscoreToCamelCase(true) in code. Hand-written resultType mappers depend on this.
- **db.auth.JwtService**: self-signed RS256 JWT issue/validate lives in the DB module so both gateways can share it. It is deliberately *not* a Spring bean — each gateway constructs its own instance with a distinct issuer/audience, so customer and admin tokens can never cross realms. Machine (service-to-service) tokens come from litemall-authserver instead.
- **Logical delete everywhere**: entities carry deleted tinyint(1); services filter deleted=0; hand-written mappers replicate the pattern manually. Gotcha: the generated andLogicalDeleted(true) matches only *deleted* rows.
- **JSON type handlers** (JsonIntegerArrayTypeHandler, JsonNodeTypeHandler ...) map JSON columns for CJ/goods enrichment; BigDecimalTypeHandler guards money precision.
- **One @MapperScan universe**: litemall-db wires all mappers from db.dao; only litemall-order additionally declares its own @MapperScan("...db.dao"). There is no module-local mapper pattern — with one exception (§7.4, the unpaid-order task table via raw JdbcTemplate).
- **db services are thin**: 51 LitemallXxxService classes wrap mappers with Example-criteria queries. Newer verticals (article/page, store, freight, outbox, CJ dispute/sourcing) skip the service layer and are consumed mapper-direct by their owning microservice.

3 · Full table inventory by domain

All 79 `litemall_*` tables in the live schema. **Owner** = the service that writes the table (the logical aggregate root); readers listed in §5. Legend:

`order` `goods-mgmt` `promotion` `loyalty` `gateway-api`
`gateway-admin` `core` `shared` `orphan`

3.1 Identity, auth & administration

Table	Rows	Owner	Purpose
<code>litemall_user</code>	3	<code>gateway-api</code> <code>shared</code>	Customer accounts. <code>gateway-api</code> registers/authenticates (V37 added <code>email</code>); order debits wallet balance (<code>now_money</code>); loyalty updates level/points; <code>goods-mgmt</code> reads for identity.
<code>litemall_user_refresh_token</code>	161	<code>gateway-api</code>	Customer refresh tokens (V15), revoked wholesale on password change.
<code>litemall_user_reset_token</code>	2	<code>gateway-api</code>	SHA-256-hashed single-use password-reset tokens, 30-min expiry (V37).
<code>litemall_admin</code>	5	<code>gateway-admin</code>	Admin accounts, BCrypt credentials.
<code>litemall_admin_refresh_token</code>	108	<code>gateway-admin</code>	Admin refresh tokens (V16).
<code>litemall_role</code> / <code>litemall_permission</code>	3 / 31	<code>gateway-admin</code>	Admin RBAC (roles relayed as <code>X-User-Roles</code>).
<code>litemall_log</code>	49	<code>gateway-admin</code>	Admin audit trail — written by <code>AdminAuditLogFilter</code> at the edge.
<code>litemall_notice</code> / <code>litemall_notice_admin</code>	0 / 7	<code>gateway-admin</code>	Admin notice authoring + per-admin inbox (read/delete scoped to the logged-in admin).
<code>litemall_user_token</code>	0	<code>orphan</code>	Legacy mini-app session tokens; superseded by JWT.

3.2 Catalog & content (goods-management, co-read by `goodsapi-analytic`)

Table	Rows	Owner	Purpose
<code>litemall_goods</code>	7 224	<code>goods-mgmt</code> <code>shared</code>	Product master. V23 added <code>source</code> (local cj) + <code>cj_pid</code> (CJ linkage); V31 ranking signals; <code>temp_id</code> binds a freight template (read by order).
<code>litemall_goods_product</code>	11 355	<code>goods-mgmt</code> <code>shared</code>	SKUs (price/stock). <code>cj_vid</code> per V23. Stock is mutated only via <code>goods-mgmt</code> 's guarded <code>reduce/restore</code> endpoints — order calls them over HTTP, never SQL.
<code>litemall_goods_attribute</code> / <code>_specification</code>	1 083 / 9 219	<code>goods-mgmt</code>	Display attributes and spec axes/values.
<code>litemall_category</code> / <code>litemall_brand</code>	737 / 49	<code>goods-mgmt</code>	Category tree (incl. 639 CJ-imported nodes, V26 widened id) and brands.
<code>litemall_comment</code>	1 004	<code>goods-mgmt</code>	Reviews; <code>admin_content</code> = one-shot seller reply (re-reply → <code>errno 622</code>).
<code>litemall_keyword</code> / <code>litemall_issue</code> / <code>litemall_topic</code> / <code>litemall_ad</code>	7/4/20/3	<code>goods-mgmt</code>	Search keywords, FAQ, curated topics, banner ads (ad admin currently via <code>gateway-admin</code> edge, flagged to migrate).

litemall_storage	4	goods-mgmt core	Uploaded-file metadata; customer uploads (magic-byte-verified) + admin storage vertical; served via /srv/storage/fetch/**.
litemall_article / litemall_article_category	2 / 0	goods-mgmt	Article CMS (V36): jsoup-sanitized MEDIUMTEXT, atomic view-count increments.
litemall_page	2	goods-mgmt	DIY home/custom pages, palette-v1 JSON ≤64KB; single active home enforced in-schema via generated-column UNIQUE index (V36).
litemall_search_history	2	goods-mgmt	Per-user search log (soft-fail write; consecutive-dupe dedupe).
litemall_cj_product	7 712	goods-mgmt	CJ Dropshipping snapshot — the OCS index source (V19/21/22/31). Sync guarded by complete-fetch gate + prune-max-fraction tripwire after the July catalog-erosion incident.
litemall_cj_sourcing_request	1	goods-mgmt	CJ sourcing-request projection (V32).
litemall_goods_description / _log / _rule	0	orphan	V14 upstream-parity tables; never wired to code.

3.3 Order, fulfillment & wallet (litemall-order)

Table	Rows	Owner	Purpose
litemall_order	64	order	Order aggregate root. Baseline columns + V27 CJ linkage (source , cj_order_id/num , address_id , country_code) + V33 cj_order_status (a column , not a separate table) + V35 pickup (delivery_type , store_id , unique verify_code , verify_time , verified_by).
litemall_order_goods	62	order	Denormalized line items (name/price/spec snapshot at purchase time — deliberately not FK-joined to live catalog).
litemall_order_status	117	order	Append-only status-transition audit/timeline (V11, V24 added old/new codes). Written in the same TX as every transition; also carries same-status hops (aftersale_approve , admin_offline_pay , writeoff , cj_sync).
litemall_event_outbox	149	order	Transactional outbox (V25): event rows inserted inside the business TX, drained to Kafka by a scheduled relay.
litemall_unpaid_order_task	0	order	Auto-cancel work queue (V20). The only table accessed outside litemall-db — raw JdbcTemplate with INSERT...ON DUPLICATE KEY and SELECT...FOR UPDATE SKIP LOCKED batch claiming.
litemall_aftersale	7	order	RMA/aftersale requests; approval settles refunds to the original tender.
litemall_cart	84	order shared	Cart lines (checked/price snapshot); order owns checkout flow, goods-mgmt self-heals CJ SKU placeholders after enrichment.
litemall_address	6	order	Shipping addresses; V29 added international codes (free-text province matched case-insensitively by freight).
litemall_store	2	order	Physical pickup stores (V35) with geo/hours/ is_show .
litemall_shipping_templates / _region / _free	2 / 2 / 0	order	Freight templates (V8, V34 intl: country_code , province_name): first/continue ladder, region specificity, free rules. Goods rows bind via litemall_goods.temp_id .

litemall_cj_dispute	0	order	CJ dispute projection per order (V28), reconciled via {order_sn}-D{n} business id.
litemall_user_bill	14	order	Wallet ledger (V3). The wallet vertical is absorbed into litemall-order (the separate wallet-service module stayed an empty placeholder); refunds are capped at the captured ledger amount, idempotent by business key.
litemall_user_recharge / litemall_user_extract	0 / 0	order	Wallet top-up and withdrawal requests (V3).

3.4 Promotion (litemall-promotion-service)

Table	Rows	Owner	Purpose
litemall_coupon / litemall_coupon_user	6 / 8	promotion shared	Coupon rules and per-user instances. Promotion owns issuance; order redeems/releases only through the promotion facade (HTTP), never SQL; goods-mgmt reads for display.
litemall_groupon / litemall_groupon_rules	0 / 3	promotion + order	Group-buy: rules owned by promotion; litemall_groupon participation rows are written by order at submit (legacy-table revive).
litemall_seckill / litemall_seckill_time	0 / 0	promotion	Flash sales (V9).
litemall_bargain / _help / _user	0	promotion	Bargain/price-chop campaigns (V10).
litemall_combination / litemall_combination_pink	6 / 11	promotion	Combination (team) deals and team/member records (V17, V30).
litemall_promotion_campaign	0	promotion	Algorithmic targeting campaigns with RFM segmentation (V18).

3.5 Loyalty & user engagement

Table	Rows	Owner	Purpose
litemall_user_level / litemall_system_user_level	0 / 0	loyalty	Member-level records and level definitions (V5).
litemall_user_integral_record / _experience_record / _sign	0	loyalty	Points, experience, and sign-in ledgers (V4).
litemall_collect / litemall_footprint / litemall_feedback	2 / 16 / 0	goods-mgmt	Favorites, browse history (also drives CJ enrichment priority), user feedback.
litemall_user_brokerage_record / litemall_user_group / litemall_user_tag	0	orphan	Distribution/brokerage and CRM segmentation (V6/V7) — deferred by Wave-4 decision, never wired.

3.6 System, region & SMS

Table	Rows	Owner	Purpose
litemall_system	18	gateway-admin core	Key-value config (litemall_mall_*, litemall_express_*, litemall_order_*). Admin edge writes; litemall-core's static SystemConfig caches per-JVM (runtime consumption by services is a documented handoff, not automatic).
litemall_region	3 231	goods-mgmt	China administrative-region tree, served via lazily-built in-memory cascade (/srv/region/**); deliberately decoupled from free-text address provinces.

litemall_system_group / _group_data / _menu	0	orphan	V12 parity tables; unused.
litemall_sms_record / litemall_sms_template	0	orphan	V13; SMS flows deferred — core NotifyService uses config + external APIs, no tables.

4 · Service → table ownership matrix

Service (module)	Runtime role	Tables WRITTEN (owned)	Tables READ from other domains
litemall-order :8085/:8086	Orders, cart, checkout, wallet, aftersale, freight, pickup stores, CJ order lifecycle	order, order_goods, order_status, event_outbox, unpaid_order_task, aftersale, cart, address, store, shipping_templates(+_region,_free), cj_dispute,groupon (participation), coupon_user (via facade only — see right), user_bill, user_recharge, user_extract, user.now_money (wallet debit)	goods + goods_product (validation/pricing snapshot; stock only via HTTP facade),groupon_rules, coupon (read; redeem via promotion facade), user
litemall-goods-management :8082	Catalog, search/OCS, content CMS, engagement, CJ catalog pipeline, region, storage	goods, goods_product, goods_attribute, goods_specification, category, brand, ad, topic, keyword, issue, search_history, feedback, collect, footprint, comment, storage, article, article_category, page, cj_product, cj_sourcing_request, coupon/coupon_user (display-side assign helpers)	order + order_goods (dashboard StatMapper — the deliberate ACL exception) , user, address, region, cart (CJ self-heal)
litemall-goodsapi-analytic	Catalog analytics + CJ enrichment sidecar	co-writes the catalog family (goods, goods_product, attribute, specification, brand, category)	cart, user, admin, role, permission (auth context, read-only)
litemall-promotion-service :8088	Coupons, groupon rules, seckill, bargain, combination, campaigns	coupon, coupon_user, groupon_rules, seckill, seckill_time, bargain(+_help,_user), combination, combination_pink, promotion_campaign	order stats only via Feign to order-service (HTTP, not SQL)
litemall-loyalty-service	Levels, points, experience, sign-in	user_level, user_experience_record, user_integral_record, user_sign; updates level/points on user	user, system_user_level
litemall-gateway-api :8090 (dev :9000/:9001)	Customer edge: JWT auth, register/reset, SPA; routes /srv/** to services	user (register/profile), user_refresh_token, user_reset_token	user (login lookup)
litemall-gateway-admin :18080	Admin edge: JWT auth, SPA, config/notice/profile edge verticals, audit	admin_refresh_token, log (audit filter), system (config), notice, notice_admin	admin, role, permission, ad
litemall-core (shared lib)	NotifyService (mail/SMS), SystemConfig, ExpressService, storage/QR helpers	storage (uploads), system (init defaults)	system (static config cache)
litemall-authserver / eureka / config / svcsecurity	Machine-token OAuth2, discovery, config, security filters	— no database access (authserver explicitly excludes the DataSource) —	

5 · Shared tables and orphan tables

5.1 Tables touched by multiple services

Table	Writer(s)	Reader(s)	Why it's safe
<code>litemall_user</code>	gateway-api (identity), order (wallet balance), loyalty (level/points)	goods-mgmt, goodsapi-analytic	Column-level partition: each writer owns disjoint columns (credentials vs <code>now_money</code> vs level/points).
<code>litemall_coupon(_user)</code>	promotion (issue), order (redeem — via HTTP facade), goods-mgmt (assign helpers)	order, goods-mgmt	State transitions on redemption go through promotion's endpoints; order holds compensation (release) not SQL.
<code>litemall_goods(_product)</code>	goods-mgmt + goodsapi-analytic (co-owners)	order (price/spec snapshot at submit)	Order copies values into <code>order_goods</code> instead of joining later; stock mutations HTTP-only with guarded atomic SQL on the goods side (errno 631/632).
<code>litemall_order(_goods)</code>	order only	goods-mgmt dashboards (StatMapper)	Read-only aggregates; see §8.2.
<code>litemall_cart</code>	order (checkout), goods-mgmt (CJ SKU self-heal)	goodsapi-analytic	Self-heal only rewrites CJ placeholder product ids post-enrichment.
<code>litemall_system</code>	gateway-admin (config UI), core (init)	core SystemConfig (all services)	Key-prefix namespacing (<code>litemall_mall_/express_/order_</code>); unknown keys rejected at the edge.
<code>litemall_storage</code>	goods-mgmt (customer uploads), core (admin storage)	public fetch route	Append-only blobs keyed by random name.

5.2 Orphan tables (in schema, zero code references)

12 tables carried for upstream/crmeb parity but never wired — candidates for a future cleanup migration, all empty:

`goods_description`, `goods_log`, `goods_rule` (V14) · `sms_record`, `sms_template` (V13) · `system_group`, `system_group_data`, `system_menu` (V12) · `user_brokerage_record` (V7) · `user_group`, `user_tag` (V6) · `user_token` (V1).

These map 1:1 to the features deliberately deferred in Wave-4 planning: brokerage/distribution, user tags/groups, SMS flows, and the legacy mini-app token store.

6 · Migration timeline (Flyway V1-V37)

V	Migration	Effect	V	Migration	Effect
1	baseline	37 upstream tables (BASELINE — not executed)	20	unpaid_order_task	auto-cancel queue

2	enrich_existing	+temp_id/weight/volume on goods, addr/cart/coupon cols	21-22	cj detail cols/html	CJ snapshot enrichment
3	wallet finance	user_bill, user_extract, user_recharge	23	cj_native_linkage	cj_pid/cj_vid onto goods tables
4-5	engagement + level	integral/experience/sign, (system_)user_level	24-25	status cols + outbox	old/new codes; event_outbox
6-7	crm + distribution	user_group/tag/token, brokerage (all unused)	26	widen cj cat id	—
8	shipping templates	3 freight tables (reused by Wave 4!)	27-28	order cj + dispute	source/cj_order_* cols; cj_dispute
9-10	seckill + bargain	5 flash-sale/bargain tables	29-31	intl addr, comb_pink, ranking	—
11	order lifecycle	order_status audit table	32-33	cj sourcing + lifecycle	sourcing_request; cj_order_status col
12-14	system/sms/goods parity	8 tables (all still orphans)	34-35	freight intl + pickup	country_code/province_name; store + verify_code
15-16	refresh tokens	customer + admin	36	content (out of order!)	article(+cat), page
17-19	combination/campaign/cj	promotion + cj_product snapshot	37	user email + reset	user.email; user_reset_token

Operational rules learned the hard way: version numbers are claimed by checking `flyway_schema_history` immediately before first boot (V17 and V32 were both taken mid-flight by sibling branches); V36 landed before V34/V35, so `out-of-order: true` is now permanent; whichever service boots first applies shared migrations; **never flyway repair**.

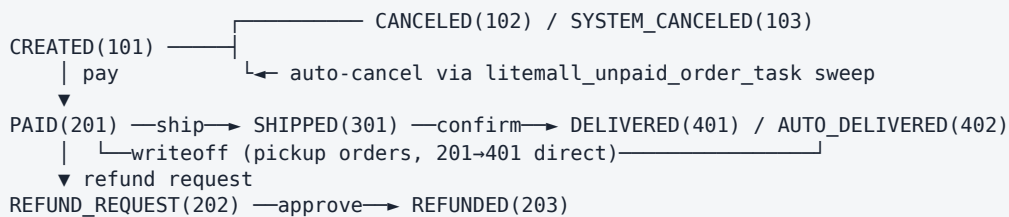
7 · Order domain deep dive

7.1 Internal DDD layout (`litemall-order`)

<code>interfaces/rest/</code>	customer + admin controllers (Order, Cart, Address, Aftersale, Wallet, CjOrder, CjDispute, Store, AdminFreight...) – DTO boundary
<code>application/</code>	<code>LiteMallOrderOrchestratorService</code> ← the <code>@Transactional</code> façade
<code>internal/</code>	<code>LiteMallOrderServiceImpl</code> (state-machine engine), service layers (Cart/Coupon/Groupon/Store, <code>FreightCalculationService</code> , <code>OrderAdminExtrasService</code>), schedulers (<code>UnpaidOrderTask</code> , <code>AutoConfirm</code> , <code>CjOrderStatusSync</code>)
<code>domain/</code>	aggregates (<code>LiteMallOrderAggregate</code> ...), repository PORTS, <code>LiteMallOrderStatus</code> enum (the transition graph), domain events
<code>infrastructure/</code>	repositories/impl (adapters over <code>litemall-db</code> mappers), services/acl/facades (Goods, Promotion, CJ), feignclients, messaging/outbox (writer + relay), acl/express + acl/printer

Dependency direction is enforced: application and domain import no adapter packages; every external system (goods, promotion, CJ, express carriers, receipt printers) sits behind a port with adapters in `infrastructure/`.

7.2 The state machine — a single transition source



- **One writer:** `LiteMallOrderServiceImpl` is the only class that mutates `order_status`. Each transition: aggregate-level `canTransitionTo` check → **guarded conditional UPDATE** (`WHERE order_status = <expected>`; 0 rows = a concurrent writer won the race → clean `IllegalStateException`, no partial state) → append a `litemall_order_status` history row in the same TX → publish domain events.
- **Timeline as data:** the history table doubles as the customer/admin-visible order timeline (`/srv/order/{id}/timeline`), including same-status annotations (`cj_sync`, `writeoff`, `admin_offline_pay`, aftersale hops).
- **Optimistic locking** (`updateWithOptimisticLocker`) protects non-status field updates.

7.3 Transactional outbox → Kafka

Domain events (`orderCreated`, `orderPaymentSucceeded`, `grouponParticipated`) are written as rows into `litemall_event_outbox` inside the business transaction, then a scheduled relay (default every 10 s) forwards PENDING rows to Kafka and marks them SENT. This guarantees an event is emitted iff the state change committed — no dual-write anomaly, and Kafka being down delays but never loses events. Rows without a Kafka binding are retained as pure audit. (Currently produce-only: no service in the repo consumes these topics yet — the subscription seam is ready for analytics/notification consumers.)

7.4 Payment tender & money conventions

pay_id shape	Meaning	Refund routing
WALLET	Paid by wallet debit (server-side, ledgered in <code>user_bill</code>)	Wallet credit, capped at captured amount, idempotent

CARD:<psp-ref> etc.	External charge, client-confirmed (Stripe PaymentIntent id)	PSP-reversal seam (logged event; no wallet credit)
OFFLINE:<ref>	Admin mark-paid (bank transfer/cash); fits the same METHOD:ref convention	Resolves automatically through the same prefix parser
null/empty	Unpaid	— (channel stats bucket it as UNPAID)

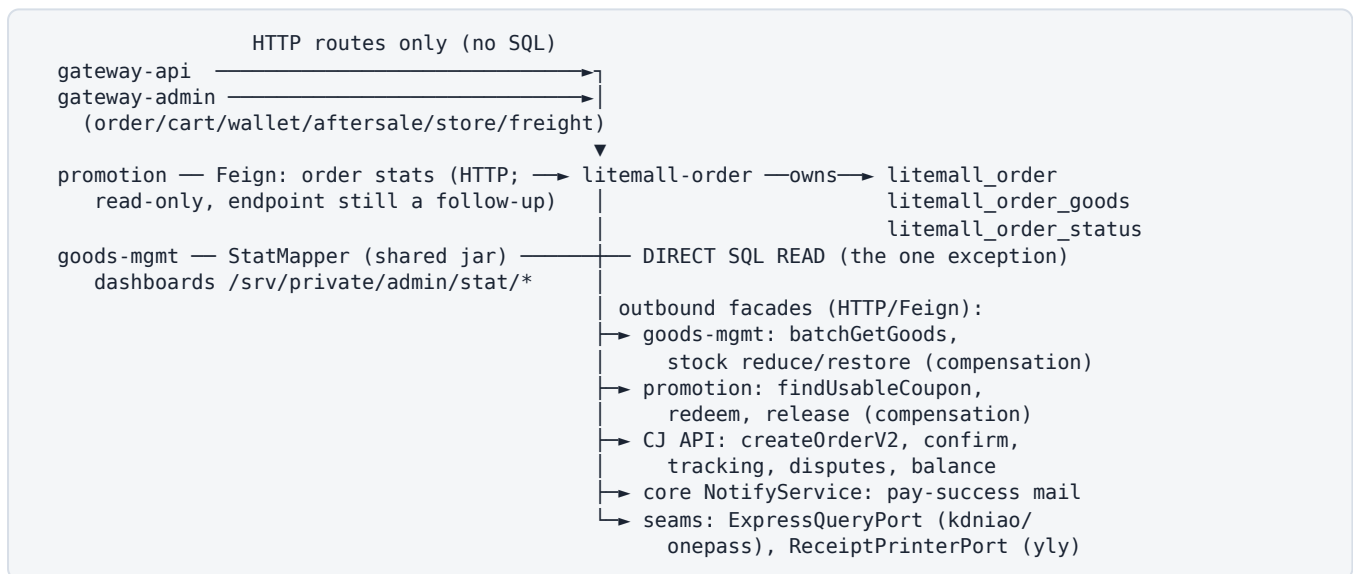
Because the tender is encoded in one column with one convention, refund-parity, the admin channel/tender dashboard (`statByTender = substring_index(pay_id, ':', 1)`), and offline payments all compose without extra tables.

7.5 CJ dropshipping & pickup as column extensions

Both verticals extend `litemall_order` rather than adding satellite order tables: CJ linkage/lifecycle lives in `source, cj_order_id, cj_order_num, cj_order_status` (V27/V33) — the CJ replay fires *after commit* of the pay transaction, and a 5-minute sweep reconciles remote status into timeline hops; pickup lives in `delivery_type, store_id, verify_code` (UNIQUE), `verify_time, verified_by` (V35) — the verify code is generated inside `markOrderPaid`'s TX (unpaid orders never carry a redeemable code), and write-off is a direct 201→401 transition guarded by `WHERE order_status=201 AND verify_time IS NULL`, so a double scan cleanly loses on zero rows.

8 · How other services interact with order tables

8.1 Interaction map



8.2 The rules, and the one exception

- **Gateways never touch order tables.** Both route `/srv/order/**`, `/srv/cart/**`, `/srv/wallet/**`, `/srv/private/admin/order|aftersale|store|freight/**` to `lb://order-service-app`. (gateway-admin's `litemall_order_` string is a `litemall_system` config-key prefix, not the table.)
- **Promotion reads order data over HTTP only** — `OrderStatisticsFeignClient` targets a customer-RFM stats endpoint on order-service (agreed contract; the order-side implementation is still a recorded follow-up, so the adapter degrades gracefully today).
- **The deliberate exception:** goods-management's admin dashboard (`AdminStatController`) calls `litemall-db's StatService → StatMapper.xml`, which aggregates `litemall_order / litemall_order_goods` (revenue where `order_status ≥ 201`) **directly in SQL**. It is read-only, aggregate-only, and cheap —

accepted as pragmatic given the shared schema, but it is the one place a service crosses the order boundary without HTTP. If order tables ever move out, this dashboard is the migration cost.

- **Order's writes into other domains are compensating façades, not SQL:** stock is reduced through goods-management's guarded endpoint (atomic `WHERE number ≥ ?` update; errno 631) and restored best-effort on cancel/refund; coupons are redeemed through promotion and released on rollback/cancel; failures map to typed exceptions (invalid coupon vs service-unavailable) so checkout degrades predictably.
- **Wallet is order-local by design:** the wallet ledger lives beside the order tables and is settled in the same transactions that need it (pay, refund) — the strongest-consistency money path avoids a distributed transaction entirely.

9 · Design principles & rationale

1. **Shared database, logical ownership.** One schema and one hand-maintained persistence library keep operations simple (single Flyway history, no cross-DB joins for stats) while table-ownership conventions + HTTP facades preserve service autonomy where it matters: *writes*. Every cross-domain write goes through the owning service's API; cross-domain reads are either snapshots (`order_goods`), HTTP, or the one documented StatMapper exception.
2. **Consistency where money moves, eventual consistency elsewhere.** Status transitions, wallet settlement, verify-code generation, and outbox writes share one ACID transaction with guarded conditional UPDATES. Cross-service effects (stock, coupons, CJ placement, receipts, notifications) happen via pre-checks before the TX, after-commit hooks, compensations, or the outbox relay — never inside the transaction where a remote failure would poison a rollback (the "422-inside-@Transactional = rollback-only 502" landmine that shaped the orchestrator's pre-check zone).
3. **Snapshot, don't join.** Order lines copy name/price/spec at purchase time; CJ products snapshot into `litemall_cj_product`; coupon amounts freeze onto the order. Historical records stay correct as the catalog mutates.
4. **Schema-enforced invariants beat application checks.** Unique `verify_code`, the generated-column unique index guaranteeing a single active DIY home, unique username/mobile closing registration races, conditional UPDATES as the concurrency arbiter — the database is the last line of defense, and the code treats 0-rows-updated/duplicate-key as first-class outcomes.
5. **Ports & adapters for every external system, disabled-by-default seams.** CJ, express carriers (kdniao/OnePass), receipt printers (Yly), and reset-mail are ports with no-op/logging defaults, selected by configuration. New integrations land as seams that cannot break the happy path (auto-print failures never block payment; tracking-provider-off returns a note, not an error).
6. **Deny-by-default at every boundary.** `svcsecurity` blocks anonymous paths unless explicitly listed per service; the gateways strip and re-assert identity headers (`X-User-Id`); customer and admin JWT realms are cryptographically separate; service-to-service calls use short-lived machine tokens from authserver.
7. **Evolution by additive migration.** The V1 baseline is frozen; every change since is an additive Flyway migration plus hand-written entity/mapper code. Existing structures are reused before new ones are minted (V8 freight tables revived six weeks later by Wave 4; the legacy aftersale/groupon tables revived rather than replaced), and deferred features' tables (V6/V7/V12-V14) wait as documented orphans rather than schema churn.

Summary. 79 tables, one schema, seven data-owning runtime modules. `litemall-db` is the shared contract; each table has exactly one writing service (with the column-partitioned `litemall_user` and the co-owned catalog as understood exceptions); order data is written only by `litemall-order` through a single-writer state machine with an audit timeline and a transactional outbox; the only direct cross-service read of order tables is the goods-management stats dashboard. Money paths are locally transactional, everything remote is a compensated or after-commit facade.